2Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения высшего образования
**«Владимирский государственный университет
Имени Александра Григорьевича и Николая Григорьевича Столетовых»
(МИВлГУ)**

Факультет ИТР

Кафедра ПИн

ЛАБОРАТОРНАЯ РАБОТА №9

по Разработка кроссплатформенных приложений

Тема Разработка контроллеров

Руководитель

Кульков Я.Ю.
(фамилия, инициалы)

(подпись) (дата)

Студент ПИн-124
(группа)

Кочедыков Е.А.
(фамилия, инициалы)

(подпись) (дата)

Муром 2026

Лабораторная работа № 9

Тема: Разработка контроллеров

Цель работы: изучить способы разработки контроллеров

Задание.

1. Для спроектированных ранее fxml-представлений пользовательского интерфейса разработать классы контроллеров.
2. Проверить и протестировать работу всего приложения с точки зрения пользователя системы.
3. Все вводимые значение должны проверяться на корректность.
4. Классы Dao должны внедряться в контроллеры через конструкторы
5. Тестовые примеры должны быть осмысленными и корректными в соответствии с предметной областью и задачей.

Ход работы:

Листинг 1. GiveMaterialController.java

```
package ru.kochedykovdev.laba7.controllers;


import javafx.beans.binding.Bindings;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.scene.control.DatePicker;
import javafx.scene.control.ListCell;
import javafx.scene.control.TextField;
import javafx.stage.Stage;
import ru.kochedykovdev.laba7.dao.BuildingObjectDao;
import ru.kochedykovdev.laba7.dao.InvoiceDao;
import ru.kochedykovdev.laba7.dao.InvoiceItemDao;
import ru.kochedykovdev.laba7.dao.MaterialCardDao;
import ru.kochedykovdev.laba7.dao.MaterialStockDao;
import ru.kochedykovdev.laba7.model.BuildingObject;
import ru.kochedykovdev.laba7.model.Invoice;
import ru.kochedykovdev.laba7.model.InvoiceItem;
import ru.kochedykovdev.laba7.model.MaterialCard;
import ru.kochedykovdev.laba7.model.MaterialStock;
```

```
import ru.kochedykovdev.laba7.util.FieldValidation;

import java.math.BigDecimal;
import java.sql.SQLException;
import java.time.LocalDateTime;
import java.util.Optional;

public class GiveMaterialController {

    private final InvoiceDao invoiceDao;
    private final InvoiceItemDao invoiceItemDao;
    private final BuildingObjectDao buildingObjectDao;
    private final MaterialCardDao materialCardDao;
    private final MaterialStockDao materialStockDao;

    @FXML
    private TextField invoiceNumberField;
    @FXML
    private ComboBox<BuildingObject> objectComboBox;
    @FXML
    private DatePicker issueDatePicker;
    @FXML
    private ComboBox<MaterialCard> materialComboBox;
    @FXML
    private TextField quantityField;
    @FXML
    private TextField priceField;
    @FXML
    private Button submitButton;

    public GiveMaterialController(InvoiceDao invoiceDao,
                                  InvoiceItemDao invoiceItemDao,
                                  BuildingObjectDao buildingObjectDao,
                                  MaterialCardDao materialCardDao,
                                  MaterialStockDao materialStockDao) {
        this.invoiceDao = invoiceDao;
        this.invoiceItemDao = invoiceItemDao;
        this.buildingObjectDao = buildingObjectDao;
        this.materialCardDao = materialCardDao;
        this.materialStockDao = materialStockDao;
    }

    @FXML
    private void initialize() {
        try {
```

```
            setupCombo(objectComboBox);
            setupCombo(materialComboBox);
            objectComboBox.getItems().setAll(buildingObjectDao.findAll());
            materialComboBox.getItems().setAll(materialCardDao.findAll());
            issueDatePicker.setValue(java.time.LocalDate.now());

            FieldValidation.onlyDecimal(quantityField);
            FieldValidation.onlyDecimal(priceField);

            FieldValidation.bindSubmit(submitButton, Bindings.createBooleanBinding(
                    () -> !invoiceNumberField.getText().isBlank()
                            && objectComboBox.getValue() != null
                            && materialComboBox.getValue() != null
                            && issueDatePicker.getValue() != null
                            && isPositive(quantityField.getText())
                            && isPositive(priceField.getText()),
                    invoiceNumberField.textProperty(),
                    objectComboBox.valueProperty(),
                    materialComboBox.valueProperty(),
                    issueDatePicker.valueProperty(),
                    quantityField.textProperty(),
                    priceField.textProperty()
            ));
        } catch (SQLException e) {
            showError(e.getMessage());
        }
    }

    @FXML
    private void onCloseClick(ActionEvent event) {
        closeWindow(event);
    }

    @FXML
    private void onSubmitClick(ActionEvent event) {
        try {
            MaterialCard material = materialComboBox.getValue();
            BigDecimal qty = new BigDecimal(quantityField.getText().trim());

            MaterialStock stock = findStock(material.getId())
                    .orElseThrow(() -> new IllegalStateException("Материала нет на
складе"));

            if (stock.getQuantity().compareTo(qty) < 0) {
```

```
                throw new IllegalStateException("Недостаточно материала на
складе");
            }

            Invoice invoice = new Invoice(
                    null,
                    invoiceNumberField.getText().trim(),
                    objectComboBox.getValue().getId(),
                    issueDatePicker.getValue()
            );
            invoiceDao.insert(invoice);

            InvoiceItem item = new InvoiceItem(
                    null,
                    invoice.getId(),
                    material.getId(),
                    qty,
                    new BigDecimal(priceField.getText().trim())
            );
            invoiceItemDao.insert(item);

            stock.setQuantity(stock.getQuantity().subtract(qty));
            stock.setLastUpdated(LocalDateTime.now());
            materialStockDao.update(stock);

            closeWindow(event);
        } catch (Exception e) {
            showError(e.getMessage());
        }
    }

    private Optional<MaterialStock> findStock(Long materialId) throws SQLException
{
        return materialStockDao.findAll().stream()
                .filter(s -> materialId.equals(s.getMaterialId()))
                .findFirst();
    }

    private static boolean isPositive(String text) {
        if (text.isBlank()) {
            return false;
        }
        return new BigDecimal(text.trim()).compareTo(BigDecimal.ZERO) > 0;
    }
```

```
    private static <T> void setupCombo(ComboBox<T> comboBox) {
        comboBox.setCellFactory(list -> new ListCell<>() {
            @Override
            protected void updateItem(T item, boolean empty) {
                super.updateItem(item, empty);
                setText(empty || item == null ? null : item.toString());
            }
        });
        comboBox.setButtonCell(comboBox.getCellFactory().call(null));
    }

    private void closeWindow(ActionEvent event) {
        Stage stage = (Stage) ((Node) event.getSource()).getScene().getWindow();
        stage.close();
    }

    private void showError(String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setHeaderText("Ошибка");
        alert.setContentText(message);
        alert.showAndWait();
    }
}
```

Листинг 2. GiveToolController.java

```
package ru.kochedykovdev.laba7.controllers;

import javafx.beans.binding.Bindings;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.scene.control.DatePicker;
import javafx.scene.control.ListCell;
import javafx.scene.control.TextField;
import javafx.stage.Stage;
import ru.kochedykovdev.laba7.dao.BuildingObjectDao;
import ru.kochedykovdev.laba7.dao.ToolDao;
import ru.kochedykovdev.laba7.dao.ToolIssueDao;
import ru.kochedykovdev.laba7.model.BuildingObject;
import ru.kochedykovdev.laba7.model.Tool;
import ru.kochedykovdev.laba7.model.ToolIssue;
```

```
import ru.kochedykovdev.laba7.util.FieldValidation;

import java.sql.SQLException;

public class GiveToolController {

    private final ToolDao toolDao;
    private final BuildingObjectDao buildingObjectDao;
    private final ToolIssueDao toolIssueDao;

    @FXML
    private ComboBox<Tool> toolComboBox;
    @FXML
    private ComboBox<BuildingObject> objectComboBox;
    @FXML
    private TextField issuedToField;
    @FXML
    private DatePicker issueDatePicker;
    @FXML
    private DatePicker returnDatePicker;
    @FXML
    private Button submitButton;

    public GiveToolController(ToolDao toolDao,
                              BuildingObjectDao buildingObjectDao,
                              ToolIssueDao toolIssueDao) {
        this.toolDao = toolDao;
        this.buildingObjectDao = buildingObjectDao;
        this.toolIssueDao = toolIssueDao;
    }

    @FXML
    private void initialize() {
        try {
            setupCombo(toolComboBox);
            setupCombo(objectComboBox);
            toolComboBox.getItems().setAll(toolDao.findAll());
            objectComboBox.getItems().setAll(buildingObjectDao.findAll());
            issueDatePicker.setValue(java.time.LocalDate.now());

            FieldValidation.bindSubmit(submitButton, Bindings.createBooleanBinding(
                    () -> toolComboBox.getValue() != null
                            && objectComboBox.getValue() != null
                            && !issuedToField.getText().isBlank()
                            && issueDatePicker.getValue() != null
```

```
                            && returnDatePicker.getValue() != null
                            &&
!returnDatePicker.getValue().isBefore(issueDatePicker.getValue()),
                    toolComboBox.valueProperty(),
                    objectComboBox.valueProperty(),
                    issuedToField.textProperty(),
                    issueDatePicker.valueProperty(),
                    returnDatePicker.valueProperty()
            ));
        } catch (SQLException e) {
            showError(e.getMessage());
        }
    }

    @FXML
    private void onCloseClick(ActionEvent event) {
        closeWindow(event);
    }

    @FXML
    private void onSubmitClick(ActionEvent event) {
        try {
            ToolIssue issue = new ToolIssue(
                    null,
                    toolComboBox.getValue().getId(),
                    objectComboBox.getValue().getId(),
                    issuedToField.getText().trim(),
                    issueDatePicker.getValue(),
                    returnDatePicker.getValue(),
                    null
            );
            toolIssueDao.insert(issue);
            closeWindow(event);
        } catch (Exception e) {
            showError(e.getMessage());
        }
    }

    private static <T> void setupCombo(ComboBox<T> comboBox) {
        comboBox.setCellFactory(list -> new ListCell<>() {
            @Override
            protected void updateItem(T item, boolean empty) {
                super.updateItem(item, empty);
                setText(empty || item == null ? null : item.toString());
            }
```

```
        });
        comboBox.setButtonCell(comboBox.getCellFactory().call(null));
    }

    private void closeWindow(ActionEvent event) {
        Stage stage = (Stage) ((Node) event.getSource()).getScene().getWindow();
        stage.close();
    }

    private void showError(String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setHeaderText("Ошибка");
        alert.setContentText(message);
        alert.showAndWait();
    }
}
```

Листинг 3. MainController.java

```
package ru.kochedykovdev.laba7.controllers;



import javafx.beans.property.SimpleStringProperty;

import javafx.collections.FXCollections;

import javafx.event.ActionEvent;

import javafx.fxml.FXML;

import javafx.fxml.FXMLLoader;

import javafx.scene.Node;

import javafx.scene.Scene;

import javafx.scene.control.Alert;

import javafx.scene.control.TableColumn;

import javafx.scene.control.TableView;

import javafx.scene.control.TextField;
```

```
import javafx.stage.Modality;

import javafx.stage.Stage;

import ru.kochedykovdev.laba7.dao.BuildingObjectDao;

import ru.kochedykovdev.laba7.dao.InvoiceDao;

import ru.kochedykovdev.laba7.dao.InvoiceItemDao;

import ru.kochedykovdev.laba7.dao.MaterialCardDao;

import ru.kochedykovdev.laba7.dao.MaterialReceiptDao;

import ru.kochedykovdev.laba7.dao.MaterialReturnDao;

import ru.kochedykovdev.laba7.dao.MaterialStockDao;

import ru.kochedykovdev.laba7.dao.ProrabDao;

import ru.kochedykovdev.laba7.dao.SupplierDao;

import ru.kochedykovdev.laba7.dao.ToolDao;

import ru.kochedykovdev.laba7.dao.ToolIssueDao;

import ru.kochedykovdev.laba7.model.BuildingObject;

import ru.kochedykovdev.laba7.model.Invoice;

import ru.kochedykovdev.laba7.model.MaterialCard;

import ru.kochedykovdev.laba7.model.MaterialReceipt;

import ru.kochedykovdev.laba7.model.MaterialReturn;

import ru.kochedykovdev.laba7.model.MaterialStock;

import ru.kochedykovdev.laba7.model.Prorab;

import ru.kochedykovdev.laba7.model.Supplier;

import ru.kochedykovdev.laba7.model.Tool;

import ru.kochedykovdev.laba7.model.ToolIssue;
```

```
import java.io.IOException;

import java.sql.SQLException;

import java.util.ArrayList;

import java.util.HashMap;

import java.util.List;

import java.util.Map;

import java.util.stream.Collectors;



public class MainController {



    private final MaterialCardDao materialCardDao;

    private final SupplierDao supplierDao;

    private final ToolDao toolDao;

    private final ProrabDao prorabDao;

    private final BuildingObjectDao buildingObjectDao;

    private final MaterialStockDao materialStockDao;

    private final MaterialReceiptDao materialReceiptDao;

    private final InvoiceDao invoiceDao;

    private final InvoiceItemDao invoiceItemDao;

    private final MaterialReturnDao materialReturnDao;

    private final ToolIssueDao toolIssueDao;
```

```
    private List<ReferenceRow> allReferenceRows = List.of();

    private List<WarehouseRow> allWarehouseRows = List.of();

    private List<ReportRow> allReportRows = List.of();



    @FXML

    private TableView<ReferenceRow> referenceTableView;

    @FXML

    private TableColumn<ReferenceRow, String> materialsColumn;

    @FXML

    private TableColumn<ReferenceRow, String> suppliersColumn;

    @FXML

    private TableColumn<ReferenceRow, String> toolsColumn;

    @FXML

    private TableColumn<ReferenceRow, String> prorabsColumn;

    @FXML

    private TableColumn<ReferenceRow, String> objectsColumn;

    @FXML

    private TextField referenceSearchField;



    @FXML

    private TableView<WarehouseRow> warehouseTableView;

    @FXML
```

```
    private TableColumn<WarehouseRow, String> stockColumn;

    @FXML

    private TableColumn<WarehouseRow, String> receiptColumn;

    @FXML

    private TableColumn<WarehouseRow, String> invoiceColumn;

    @FXML

    private TableColumn<WarehouseRow, String> returnColumn;

    @FXML

    private TextField warehouseSearchField;



    @FXML

    private TableView<ReportRow> reportTableView;

    @FXML

    private TableColumn<ReportRow, String> movementColumn;

    @FXML

    private TableColumn<ReportRow, String> debtorsColumn;

    @FXML

    private TextField reportSearchField;



    public MainController(MaterialCardDao materialCardDao,

                          SupplierDao supplierDao,

                          ToolDao toolDao,
```

```
                           ProrabDao prorabDao,
                           BuildingObjectDao buildingObjectDao,
                           MaterialStockDao materialStockDao,
                           MaterialReceiptDao materialReceiptDao,
                           InvoiceDao invoiceDao,
                           InvoiceItemDao invoiceItemDao,
                           MaterialReturnDao materialReturnDao,
                           ToolIssueDao toolIssueDao) {
        this.materialCardDao = materialCardDao;
        this.supplierDao = supplierDao;
        this.toolDao = toolDao;
        this.prorabDao = prorabDao;
        this.buildingObjectDao = buildingObjectDao;
        this.materialStockDao = materialStockDao;
        this.materialReceiptDao = materialReceiptDao;
        this.invoiceDao = invoiceDao;
        this.invoiceItemDao = invoiceItemDao;
        this.materialReturnDao = materialReturnDao;
        this.toolIssueDao = toolIssueDao;
    }

    @FXML
    private void initialize() {
```

```
        materialsColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().materials()));

        suppliersColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().suppliers()));

        toolsColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().tools()));

        prorabsColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().prorabs()));

        objectsColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().objects()));



        stockColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().stock()));

        receiptColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().receipt()));

        invoiceColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().invoice()));

        returnColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().materialReturn()));



        movementColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().movement()));

        debtorsColumn.setCellValueFactory(row -> new
SimpleStringProperty(row.getValue().debtors()));



        refreshAll();

    }
```

```
    @FXML
    private void onRefreshReferenceClick() {
        try {
            loadReferenceRows();
            applyReferenceFilter();
        } catch (SQLException e) {
            showError(e.getMessage());
        }
    }

    @FXML
    private void onSearchReferenceClick() {
        applyReferenceFilter();
    }

    @FXML
    private void onRefreshWarehouseClick() {
        try {
            loadWarehouseRows();
            applyWarehouseFilter();
        } catch (SQLException e) {
            showError(e.getMessage());
```

```
        }
    }

    @FXML
    private void onSearchWarehouseClick() {
        applyWarehouseFilter();
    }

    @FXML
    private void onRefreshReportClick() {
        try {
            loadReportRows();
            applyReportFilter();
        } catch (SQLException e) {
            showError(e.getMessage());
        }
    }

    @FXML
    private void onSearchReportClick() {
        applyReportFilter();
    }
```

```
    @FXML
    private void onPrihodClick(ActionEvent event) {
        openDialog("prihod-view.fxml", "Приход материала", event,
PrihodController.class);
    }

    @FXML
    private void onGiveMaterialClick(ActionEvent event) {
        openDialog("give-material-view.fxml", "Выдача материала", event,
GiveMaterialController.class);
    }

    @FXML
    private void onGiveToolClick(ActionEvent event) {
        openDialog("give-tool-view.fxml", "Выдача инструмента", event,
GiveToolController.class);
    }

    @FXML
    private void onReturnToolClick(ActionEvent event) {
        openDialog("return-tool-view.fxml", "Возврат инструмента", event,
ReturnToolController.class);
    }
```

```
    private void refreshAll() {

        try {

            loadReferenceRows();

            loadWarehouseRows();

            loadReportRows();

            applyReferenceFilter();

            applyWarehouseFilter();

            applyReportFilter();

        } catch (SQLException e) {

            showError(e.getMessage());

        }

    }



    private void loadReferenceRows() throws SQLException {

        List<MaterialCard> materials = materialCardDao.findAll();

        List<Supplier> suppliers = supplierDao.findAll();

        List<Tool> tools = toolDao.findAll();

        List<Prorab> prorabs = prorabDao.findAll();

        List<BuildingObject> objects = buildingObjectDao.findAll();



        int max = Math.max(materials.size(),

                Math.max(suppliers.size(),

                        Math.max(tools.size(),
```

```
                                Math.max(prorabs.size(), objects.size())));


        List<ReferenceRow> rows = new ArrayList<>();
        for (int i = 0; i < max; i++) {
            rows.add(new ReferenceRow(
                    nameAt(materials, i),
                    nameAt(suppliers, i),
                    nameAt(tools, i),
                    nameAt(prorabs, i),
                    nameAt(objects, i)
            ));
        }
        allReferenceRows = rows;
    }


    private void loadWarehouseRows() throws SQLException {
        Map<Long, String> materialNames = new HashMap<>();
        for (MaterialCard card : materialCardDao.findAll()) {
            materialNames.put(card.getId(), card.getName());
        }

        List<String> stocks = materialStockDao.findAll().stream()
```

```
                .map(s -> formatStock(s, materialNames))
                .toList();
        List<String> receipts = materialReceiptDao.findAll().stream()
                .map(r -> formatReceipt(r, materialNames))
                .toList();
        List<String> invoices = invoiceDao.findAll().stream()
                .map(this::formatInvoice)
                .toList();
        List<String> returns = materialReturnDao.findAll().stream()
                .map(r -> formatReturn(r, materialNames))
                .toList();

        int max = Math.max(stocks.size(), Math.max(receipts.size(),
Math.max(invoices.size(), returns.size())));
        List<WarehouseRow> rows = new ArrayList<>();
        for (int i = 0; i < max; i++) {
            rows.add(new WarehouseRow(
                    at(stocks, i),
                    at(receipts, i),
                    at(invoices, i),
                    at(returns, i)
            ));
        }
```

```
        allWarehouseRows = rows;

    }



    private void loadReportRows() throws SQLException {

        Map<Long, String> materialNames = new HashMap<>();

        for (MaterialCard card : materialCardDao.findAll()) {

            materialNames.put(card.getId(), card.getName());

        }

        Map<Long, String> toolNames = new HashMap<>();

        for (Tool tool : toolDao.findAll()) {

            toolNames.put(tool.getId(), tool.getName());

        }



        List<String> movements = materialReceiptDao.findAll().stream()

                .map(r -> "Приход: " +
materialNames.getOrDefault(r.getMaterialId(), "?")

                        + " " + r.getQuantity() + " (" + r.getReceiptDate() + ")")

                .toList();



        List<String> debtors = toolIssueDao.findAll().stream()

                .filter(i -> i.getActualReturnDate() == null)

                .map(i -> toolNames.getOrDefault(i.getToolId(), "инструмент")

                        + " → " + i.getIssuedTo() + " до " + i.getReturnDate())
```

```
                .toList();


        int max = Math.max(movements.size(), debtors.size());
        List<ReportRow> rows = new ArrayList<>();
        for (int i = 0; i < max; i++) {
            rows.add(new ReportRow(at(movements, i), at(debtors, i)));
        }
        allReportRows = rows;
    }


    private void applyReferenceFilter() {
        String filter = referenceSearchField.getText().trim().toLowerCase();
        if (filter.isEmpty()) {

referenceTableView.setItems(FXCollections.observableArrayList(allReferenceRows));
            return;
        }
        referenceTableView.setItems(FXCollections.observableArrayList(
                allReferenceRows.stream()
                        .filter(row -> row.matches(filter))
                        .collect(Collectors.toList())
        ));
    }
```

```
    private void applyWarehouseFilter() {
        String filter = warehouseSearchField.getText().trim().toLowerCase();
        if (filter.isEmpty()) {
warehouseTableView.setItems(FXCollections.observableArrayList(allWarehouseRows));
            return;
        }
        warehouseTableView.setItems(FXCollections.observableArrayList(
                allWarehouseRows.stream()
                        .filter(row -> row.matches(filter))
                        .collect(Collectors.toList())
        ));
    }

    private void applyReportFilter() {
        String filter = reportSearchField.getText().trim().toLowerCase();
        if (filter.isEmpty()) {
reportTableView.setItems(FXCollections.observableArrayList(allReportRows));
            return;
        }
        reportTableView.setItems(FXCollections.observableArrayList(
                allReportRows.stream()
```

```
                        .filter(row -> row.matches(filter))
                        .collect(Collectors.toList())
        ));
    }

    private String formatStock(MaterialStock stock, Map<Long, String> names) {
        return names.getOrDefault(stock.getMaterialId(), "?") + ": " +
stock.getQuantity();
    }

    private String formatReceipt(MaterialReceipt receipt, Map<Long, String> names)
{
        return names.getOrDefault(receipt.getMaterialId(), "?") + " "
                + receipt.getQuantity() + " (" + receipt.getReceiptDate() + ")";
    }

    private String formatInvoice(Invoice invoice) {
        return "№" + invoice.getInvoiceNumber() + " (" + invoice.getIssueDate() +
")";
    }

    private String formatReturn(MaterialReturn materialReturn, Map<Long, String>
names) {
        return names.getOrDefault(materialReturn.getMaterialId(), "?") + " "
```

```
                + materialReturn.getQuantity() + " (" +
materialReturn.getReturnDate() + ")";

    }



    private static String nameAt(List<? extends Object> list, int index) {

        if (index >= list.size()) {

            return "";

        }

        Object item = list.get(index);

        if (item instanceof MaterialCard c) {

            return c.getName();

        }

        if (item instanceof Supplier s) {

            return s.getName();

        }

        if (item instanceof Tool t) {

            return t.toString();

        }

        if (item instanceof Prorab p) {

            return p.getName();

        }

        if (item instanceof BuildingObject o) {

            return o.getName();
```

```
        }

        return "";

    }



    private static String at(List<String> list, int index) {

        return index < list.size() ? list.get(index) : "";

    }



    private void openDialog(String fxml, String title, ActionEvent event, Class<?>
controllerClass) {

        try {

            FXMLLoader loader = new
FXMLLoader(getClass().getResource("/ru/kochedykovdev/laba7/" + fxml));

            loader.setControllerFactory(clazz -> createDialogController(clazz,
controllerClass));



            Scene scene = new Scene(loader.load());

            Stage stage = new Stage();

            stage.setTitle(title);

            stage.setScene(scene);

            stage.initModality(Modality.WINDOW_MODAL);

            stage.initOwner(((Stage) ((Node)
event.getSource()).getScene().getWindow()));

            stage.showAndWait();
```

```
            refreshAll();
        } catch (IOException e) {
            showError(e.getMessage());
        }
    }

    private Object createDialogController(Class<?> clazz, Class<?> expected) {
        if (clazz != expected) {
            throw new IllegalStateException("Неизвестный контроллер: " + clazz);
        }
        if (clazz == PrihodController.class) {
            return new PrihodController(materialCardDao, supplierDao,
materialReceiptDao, materialStockDao);
        }
        if (clazz == GiveMaterialController.class) {
            return new GiveMaterialController(invoiceDao, invoiceItemDao,
buildingObjectDao, materialCardDao, materialStockDao);
        }
        if (clazz == GiveToolController.class) {
            return new GiveToolController(toolDao, buildingObjectDao,
toolIssueDao);
        }
        if (clazz == ReturnToolController.class) {
            return new ReturnToolController(toolIssueDao, toolDao);
```

```
        }
        throw new IllegalStateException("Неизвестный контроллер: " + clazz);
    }

    private void showError(String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setHeaderText("Ошибка");
        alert.setContentText(message);
        alert.showAndWait();
    }

    public record ReferenceRow(String materials, String suppliers, String tools,
String prorabs, String objects) {
        boolean matches(String filter) {
            return materials.toLowerCase().contains(filter)
                    || suppliers.toLowerCase().contains(filter)
                    || tools.toLowerCase().contains(filter)
                    || prorabs.toLowerCase().contains(filter)
                    || objects.toLowerCase().contains(filter);
        }
    }
```

```
    public record WarehouseRow(String stock, String receipt, String invoice, String
materialReturn) {

        boolean matches(String filter) {

            return stock.toLowerCase().contains(filter)

                    || receipt.toLowerCase().contains(filter)

                    || invoice.toLowerCase().contains(filter)

                    || materialReturn.toLowerCase().contains(filter);

        }

    }



    public record ReportRow(String movement, String debtors) {

        boolean matches(String filter) {

            return movement.toLowerCase().contains(filter)

                    || debtors.toLowerCase().contains(filter);

        }

    }

}
```

Листинг 4. PrihodController.java

```
package ru.kochedykovdev.laba7.controllers;

import javafx.beans.binding.Bindings;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
```

```
import javafx.scene.control.ComboBox;
import javafx.scene.control.DatePicker;
import javafx.scene.control.ListCell;
import javafx.scene.control.TextField;
import javafx.stage.Stage;
import ru.kochedykovdev.laba7.dao.MaterialCardDao;
import ru.kochedykovdev.laba7.dao.MaterialReceiptDao;
import ru.kochedykovdev.laba7.dao.MaterialStockDao;
import ru.kochedykovdev.laba7.dao.SupplierDao;
import ru.kochedykovdev.laba7.model.MaterialCard;
import ru.kochedykovdev.laba7.model.MaterialReceipt;
import ru.kochedykovdev.laba7.model.MaterialStock;
import ru.kochedykovdev.laba7.model.Supplier;
import ru.kochedykovdev.laba7.util.FieldValidation;

import java.math.BigDecimal;
import java.sql.SQLException;
import java.time.LocalDateTime;
import java.util.Optional;

public class PrihodController {

    private final MaterialCardDao materialCardDao;
    private final SupplierDao supplierDao;
    private final MaterialReceiptDao materialReceiptDao;
    private final MaterialStockDao materialStockDao;

    @FXML
    private ComboBox<MaterialCard> materialComboBox;
    @FXML
    private ComboBox<Supplier> supplierComboBox;
    @FXML
    private TextField quantityField;
    @FXML
    private TextField priceField;
    @FXML
    private DatePicker receiptDatePicker;
    @FXML
    private Button submitButton;

    public PrihodController(MaterialCardDao materialCardDao,
                            SupplierDao supplierDao,
                            MaterialReceiptDao materialReceiptDao,
                            MaterialStockDao materialStockDao) {
        this.materialCardDao = materialCardDao;
```

```
        this.supplierDao = supplierDao;
        this.materialReceiptDao = materialReceiptDao;
        this.materialStockDao = materialStockDao;
    }

    @FXML
    private void initialize() {
        try {
            setupCombo(materialComboBox);
            setupCombo(supplierComboBox);
            materialComboBox.getItems().setAll(materialCardDao.findAll());
            supplierComboBox.getItems().setAll(supplierDao.findAll());
            receiptDatePicker.setValue(java.time.LocalDate.now());

            FieldValidation.onlyDecimal(quantityField);
            FieldValidation.onlyDecimal(priceField);

            FieldValidation.bindSubmit(submitButton, Bindings.createBooleanBinding(
                    () -> materialComboBox.getValue() != null
                            && supplierComboBox.getValue() != null
                            && !quantityField.getText().isBlank()
                            && !priceField.getText().isBlank()
                            && receiptDatePicker.getValue() != null
                            && isPositive(quantityField.getText())
                            && isPositive(priceField.getText()),
                    materialComboBox.valueProperty(),
                    supplierComboBox.valueProperty(),
                    quantityField.textProperty(),
                    priceField.textProperty(),
                    receiptDatePicker.valueProperty()
            ));
        } catch (SQLException e) {
            showError(e.getMessage());
        }
    }

    @FXML
    private void onCloseClick(ActionEvent event) {
        closeWindow(event);
    }

    @FXML
    private void onSubmitClick(ActionEvent event) {
        try {
            MaterialCard material = materialComboBox.getValue();
```

```
            BigDecimal qty = new BigDecimal(quantityField.getText().trim());

            MaterialReceipt receipt = new MaterialReceipt(
                    null,
                    material.getId(),
                    supplierComboBox.getValue().getId(),
                    qty,
                    new BigDecimal(priceField.getText().trim()),
                    receiptDatePicker.getValue()
            );
            materialReceiptDao.insert(receipt);

            updateStock(material.getId(), qty);
            closeWindow(event);
        } catch (Exception e) {
            showError(e.getMessage());
        }
    }

    private void updateStock(Long materialId, BigDecimal qty) throws SQLException {
        Optional<MaterialStock> existing = materialStockDao.findAll().stream()
                .filter(s -> materialId.equals(s.getMaterialId()))
                .findFirst();

        if (existing.isPresent()) {
            MaterialStock stock = existing.get();
            stock.setQuantity(stock.getQuantity().add(qty));
            stock.setLastUpdated(LocalDateTime.now());
            materialStockDao.update(stock);
        } else {
            MaterialStock stock = new MaterialStock(null, materialId, qty,
LocalDateTime.now());
            materialStockDao.insert(stock);
        }
    }

    private static boolean isPositive(String text) {
        if (text.isBlank()) {
            return false;
        }
        return new BigDecimal(text.trim()).compareTo(BigDecimal.ZERO) > 0;
    }

    private static <T> void setupCombo(ComboBox<T> comboBox) {
        comboBox.setCellFactory(list -> new ListCell<>() {
```

```
            @Override
            protected void updateItem(T item, boolean empty) {
                super.updateItem(item, empty);
                setText(empty || item == null ? null : item.toString());
            }
        });
        comboBox.setButtonCell(comboBox.getCellFactory().call(null));
    }

    private void closeWindow(ActionEvent event) {
        Stage stage = (Stage) ((Node) event.getSource()).getScene().getWindow();
        stage.close();
    }

    private void showError(String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setHeaderText("Ошибка");
        alert.setContentText(message);
        alert.showAndWait();
    }
}
```

Листинг 5. ReturnToolController.java

```
package ru.kochedykovdev.laba7.controllers;

import javafx.beans.binding.Bindings;
import javafx.event.ActionEvent;
import javafx.fxml.FXML;
import javafx.scene.Node;
import javafx.scene.control.Alert;
import javafx.scene.control.Button;
import javafx.scene.control.ComboBox;
import javafx.scene.control.DatePicker;
import javafx.scene.control.ListCell;
import javafx.scene.control.TextField;
import javafx.stage.Stage;
import ru.kochedykovdev.laba7.dao.ToolDao;
import ru.kochedykovdev.laba7.dao.ToolIssueDao;
import ru.kochedykovdev.laba7.model.Tool;
import ru.kochedykovdev.laba7.model.ToolIssue;
import ru.kochedykovdev.laba7.util.FieldValidation;

import java.sql.SQLException;
import java.util.HashMap;
```

```
import java.util.Map;

public class ReturnToolController {

    private final ToolIssueDao toolIssueDao;
    private final ToolDao toolDao;

    private final Map<Long, String> toolNames = new HashMap<>();

    @FXML
    private ComboBox<ToolIssue> toolIssueComboBox;
    @FXML
    private DatePicker actualReturnDatePicker;
    @FXML
    private TextField conditionField;
    @FXML
    private Button submitButton;

    public ReturnToolController(ToolIssueDao toolIssueDao, ToolDao toolDao) {
        this.toolIssueDao = toolIssueDao;
        this.toolDao = toolDao;
    }

    @FXML
    private void initialize() {
        try {
            for (Tool tool : toolDao.findAll()) {
                toolNames.put(tool.getId(), tool.getName());
            }

            toolIssueComboBox.setCellFactory(list -> new ListCell<>() {
                @Override
                protected void updateItem(ToolIssue item, boolean empty) {
                    super.updateItem(item, empty);
                    setText(empty || item == null ? null : formatIssue(item));
                }
            });

toolIssueComboBox.setButtonCell(toolIssueComboBox.getCellFactory().call(null));

            toolIssueComboBox.getItems().setAll(
                    toolIssueDao.findAll().stream()
                            .filter(i -> i.getActualReturnDate() == null)
                            .toList()
            );
```

```
            actualReturnDatePicker.setValue(java.time.LocalDate.now());

            FieldValidation.bindSubmit(submitButton, Bindings.createBooleanBinding(
                    () -> toolIssueComboBox.getValue() != null
                            && actualReturnDatePicker.getValue() != null
                            && !conditionField.getText().isBlank(),
                    toolIssueComboBox.valueProperty(),
                    actualReturnDatePicker.valueProperty(),
                    conditionField.textProperty()
            ));
        } catch (SQLException e) {
            showError(e.getMessage());
        }
    }

    @FXML
    private void onCloseClick(ActionEvent event) {
        closeWindow(event);
    }

    @FXML
    private void onSubmitClick(ActionEvent event) {
        try {
            ToolIssue issue = toolIssueComboBox.getValue();
            issue.setActualReturnDate(actualReturnDatePicker.getValue());
            toolIssueDao.update(issue);

            Tool tool = toolDao.findById(issue.getToolId())
                    .orElseThrow(() -> new IllegalStateException("Инструмент не
найден"));
            tool.setCondition(conditionField.getText().trim());
            toolDao.update(tool);

            closeWindow(event);
        } catch (Exception e) {
            showError(e.getMessage());
        }
    }

    private String formatIssue(ToolIssue issue) {
        String name = toolNames.getOrDefault(issue.getToolId(), "инструмент");
        return name + " — " + issue.getIssuedTo();
    }

    private void closeWindow(ActionEvent event) {
```

```
        Stage stage = (Stage) ((Node) event.getSource()).getScene().getWindow();
        stage.close();
    }

    private void showError(String message) {
        Alert alert = new Alert(Alert.AlertType.ERROR);
        alert.setHeaderText("Ошибка");
        alert.setContentText(message);
        alert.showAndWait();
    }
}
```

Вывод: В ходе лабораторной работы я изучил способы разработки контроллеров